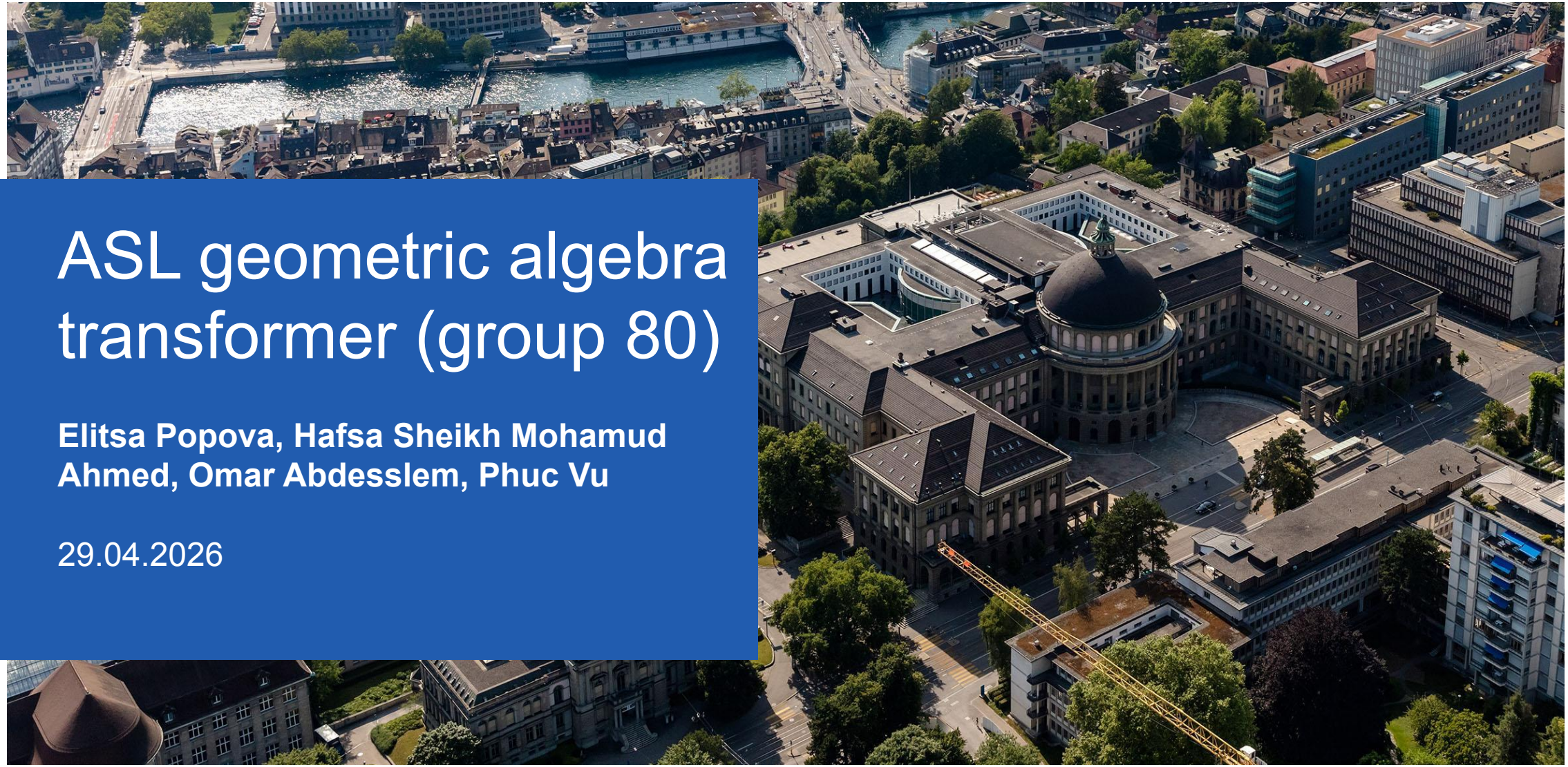
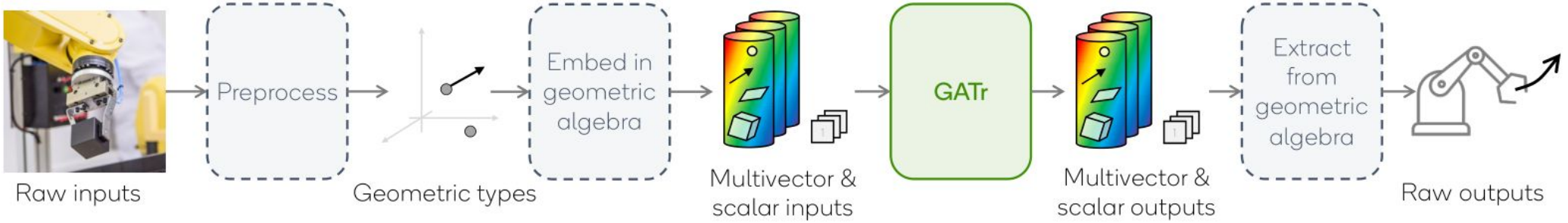


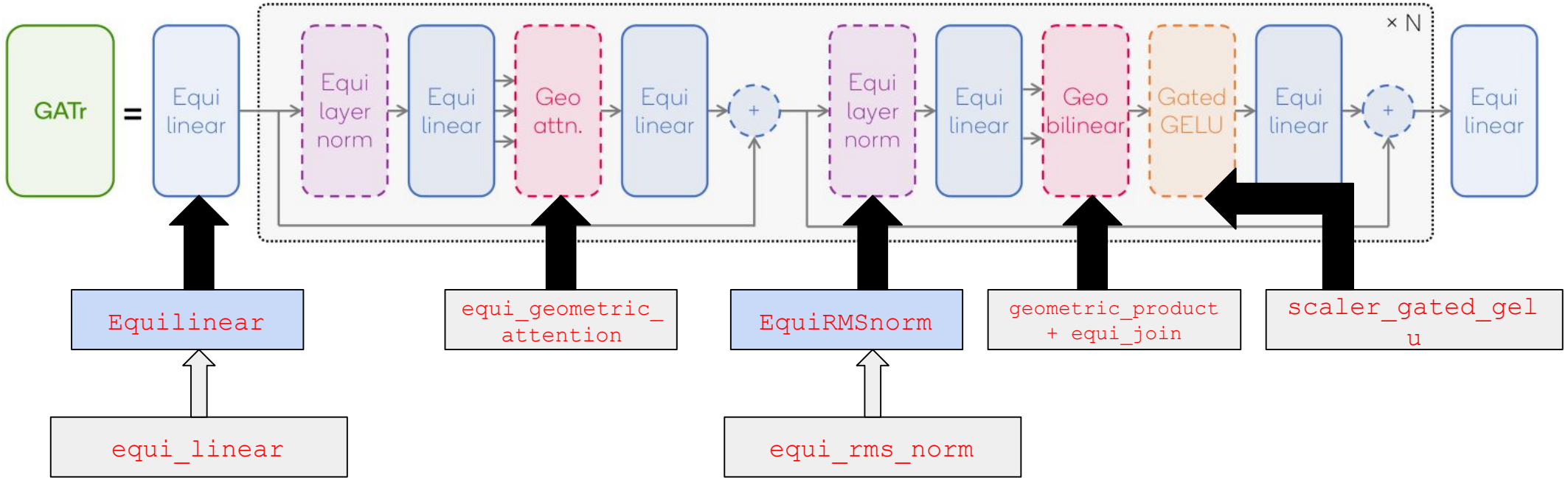
ASL geometric algebra transformer (group 80)

Elitsa Popova, Hafsa Sheikh Mohamud Ahmed, Omar Abdesslem, Phuc Vu

29.04.2026

Algorithm overview





Benchmark is the forward pass of **MVOnlyGATrModel**
fixed config: **B=8**, **N=256**, **channels=2**, **multivector dim 16**

Our C implementation

Layout

cpp/ **libtorch only used at boundary**
linear.cpp `geometric_product, outer_product, equi_linear`
dual.cpp `equi_dual, equi_join`
norm.cpp `equi_rms_norm, scaler_gated_gelu`
attention.cpp
`equi_geometric_attention`

basis/
***_cpp.pt** — precomputed (16,16,16) bilinear bases, generated by `make_basis.py`, cached as raw float* (basis for dual to be added)

ezgatr_extensions/
asl_mv_only_gatr.py swaps EzGATr ops in __init__
equilinear.py `ASLEquiLinear` only override forward to dispatch to our C kernel
asl_norm.py `ASLEquiRMSNorm` only override forward to dispatch to our C kernel

Baseline kernel/boundary shape

C++ kernel (snippet)

```cpp
for (int64_t b = 0; b < batch_size; ++b) {
    for (int i = 0; i < 16; ++i) {
        float sum = 0.0f;
        for (int j = 0; j < 16; ++j) {
            for (int k = 0; k < 16; ++k) {
                int basis_idx = i * 256 + j * 16 + k;  // 16*16 = 256
                float basis_val = basis_ptr[basis_idx];
                float x_val = x_ptr[b * 16 + j];
                float y_val = y_ptr[b * 16 + k];
                sum += basis_val * x_val * y_val;
            }
        }
        result_ptr[b * 16 + i] = sum;
    }
}
```

Torch handling at boundary (snippet)

```cpp
auto x_cpu = x.contiguous().to(torch::kCPU, torch::kFloat32);
auto y_cpu = y.contiguous().to(torch::kCPU, torch::kFloat32);
const float* x_ptr = x_cpu.data_ptr<float>();
const float* y_ptr = y_cpu.data_ptr<float>();

// Load basis
float* basis_ptr = _load_bilinear_basis_raw(Kind::OP);

// Allocate result array
int64_t batch_size = x_cpu.numel() / 16;
auto result_cpu = torch::zeros_like(x_cpu);
float* result_ptr = result_cpu.data_ptr<float>();
```

Validation

Tested code using different Matrix sizes (up to 8 batches and 256 sequence length) , compared it with Python results

```
========================================
  EzGATr End-to-End Validation Test
========================================

[INFO] Comparing Python EzGATr with ASL/C++ EzGATr
[INFO] If needed, run: python3 -m ezgatr_extensions.make_basis

[Passed] (B=1, T=4, C=2) max_err=2.98e-08,mean_err=3.80e-09
[Passed] (B=1, T=8, C=2) max_err=5.63e-07,mean_err=2.32e-08
[Passed] (B=1, T=16, C=2) max_err=5.63e-07,mean_err=2.36e-08
[Passed] (B=2, T=4, C=2) max_err=6.47e-07,mean_err=1.45e-08
[Passed] (B=2, T=16, C=2) max_err=1.42e-06,mean_err=2.77e-08
[Passed] (B=4, T=32, C=2) max_err=5.03e-05,mean_err=2.10e-07
[Passed] (B=8, T=64, C=2) max_err=1.34e-02,mean_err=1.00e-05
[Passed] (B=8, T=128, C=2) max_err=2.86e-02,mean_err=1.63e-05
[Passed] (B=8, T=256, C=2) max_err=2.77e-02,mean_err=2.91e-05
[Passed] (B=1, T=4, C=1) max_err=5.96e-08,mean_err=3.89e-09
```

```
========================================
  EzGATr equi_geometric_attention Validation
========================================

[INFO] Comparing Python equi_geometric_attention with ASL/C++ version

[Passed] (B=1, H=4, T=4, C=32) max_err=1.93e-05,mean_err=3.00e-07
[Passed] (B=1, H=4, T=16, C=32) max_err=6.96e-05,mean_err=4.31e-07
[Passed] (B=2, H=4, T=32, C=32) max_err=4.18e-03,mean_err=6.94e-06
[Passed] (B=8, H=4, T=64, C=32) max_err=4.54e-03,mean_err=2.20e-06
[Passed] (B=8, H=4, T=128, C=32) max_err=3.77e-03,mean_err=2.06e-06
[Passed] (B=8, H=4, T=256, C=32) max_err=3.02e-02,mean_err=3.09e-06
```

Timing infrastructure and cost function

Script	What it measures	Mechanism
baseline_c_code.py / baseline_python_code.py	end-to-end wall time (ms)	time.perf_counter_ns() around model(x)
baseline_c_cycles.py / baseline_python_cycles.py	end-to-end cycles	__rdtscp
baseline_per_kernel_cycles.py per-kernel cycles	per-kernel cycles	CYCLE_SCOPE macro inside each C++ kernel (?)

WARMUP = 1
REPEATS = 3, timed passes & take the avg/min/max
TEST_CASES = [(8, T, 2) for T in [16, 32, 64, 128, 256, 512, 1024, 2048, 4096]]
Only vary T, since T is the only axis driving non-linear cost

Compiler flags and cost function

```
_BASE_FLAGS = [
    "-std=c++17",
    "-O3",
    "-ffast-math",
    "-DNDEBUG",
    "-funroll-loops",
]
```

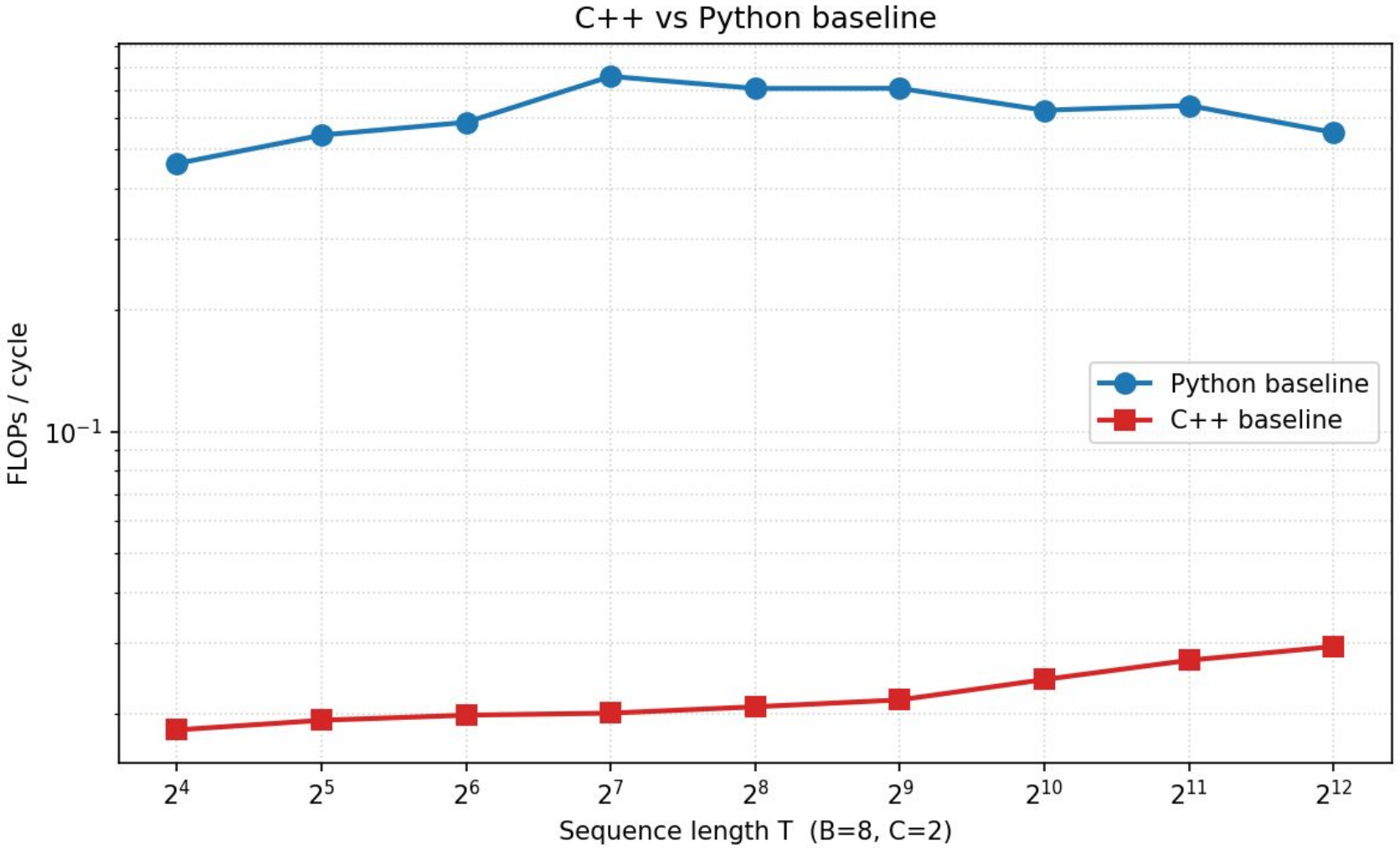
Cost function

flops_total = unweighted sum: every op = 1 FLOP.

weighted_flops = sum scaled by op cost

Op Weight
add 1
mul 1
pow2 1
div 4
sqrt 8
exp 20
tanh 20
currently they reflect rough x86 latency ratios

Performance plots



* Turbo boost not disabled
** FLOPS outside of kernel computations considered negligible

Bottleneck analysis/profiling

We measured cache misses for each of the modules/functions separately to see which ones have the most cache misses. here sorted from most to least number of cache misses:

1 equi_geometric_attention

2. equi_join

3. geometric_product

4. asl_equi_linear

5. equi_rms_norm

6. equi_dual

Optimization plan

Functions to optimize:

- `equi_linear`
- `equi_join`
- `equi_geometric_attention`
- `geometric_product`

Optimization plan

- Most of the bottleneck is in the geometric calculations, inside the raw scaled dot-product attention loop : equi_linear

  Where:

$$\text{cost} \approx B \times H \times Tq \times Tk \times (D + Dv)$$

  Where B is batch size, H number of attention heads, Tq Query sequence length, Tk Key/value sequence length, D

- Globally, for our code, we'll be adding GPU support ?

Optimization plan

- **equi_linear:** Optimization Plan

out[b,t,o,d] = Σ iΣwΣs weight[o,i,w] · basis[w,d,s] · x[b,t,i,s]

A lot of computation: channels_in × 9 × 16 iterations (per output), computation likely bottleneck

-Exploit sparsity (basis tensor sparse), basis matrix has density of 24 / 2304 = 1 / 96 ≈ 1.04%

- Scalar replacement (weight[o,i,w] outside inner loops)

-Exploit ILP (independent accumulators instead of a single running sum)

-SIMD vectorization

Optimization plan

- **equi_join:** Formula

$A = \Sigma_i\ a_i\ e_i$
$B = \Sigma\square\ b\square\ e\square$

=> J = equi_join(A, B) where:

$$J[k] = \Sigma_i\ \Sigma\square\ c[i,j,k] \cdot a[i] \cdot b[j]$$

Optimization plan

- **equi_join:** Optimization Plan

- Exploit antisymmetry: combine mirrored terms.
- Unroll fixed 16-blade formulas.
- SIMD/vectorize across batch-token multivectors.

Optimization plan

- **equi_geometric_attention:** Formula

For each batch b, head h, query token i, and key token j:

score[i,j] =

scale × (scalar_q[i] · scalar_k[j] + Σ_kind weight[kind] × φ_kind(q_mv[i]) · φ_kind(k_mv[j]))

Optimization plan

- **equi_geometric_attention**

- compute scalar + IPA + DAA dot directly, instead of using q_cat/k_cat temporaries.
- Parallelize single threaded loops, like independent B × H × Tq attention rows.
- SIMD/unroll dot products and value accumulation over D and Dv.
- Tile/cache-block over Tk and Dv for better cache locality.

Optimization plan

- **geometric_product:** Formula

$$A = \Sigma_i\, a_i\, e_i$$
$$B = \Sigma\square\, b\square\, e\square$$

$$\Rightarrow \quad A \,.\, B = \Sigma_i\, \Sigma\square\, a_i\, b\square\, (e_i\, e\square)$$

Optimization plan

- **geometric_product:** ⇒ $A \,.\, B = \Sigma_i \Sigma_j a_i b_j (e_i e_j)$

- Using sparse nonzero term list, Maybe CSR, CSL.
- Unroll fixed-size blade loops or generate formulas.
- SIMD/vectorize across batch-token multivectors.

ETH zürich